



PROCESO DIRECCIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

INTRODUCCIÓN

Las bases de datos son un elemento fundamental en el entorno informático, en la actualidad tienen múltiples aplicaciones prácticas en prácticamente todos los campos; siendo una de las herramientas de mayor utilidad para todas las disciplinas y áreas de aplicación donde existe la necesidad de gestionar datos. Los datos son cada día más voluminosos debido a la cantidad de información que circula en los sistemas existentes y su grado de precisión es también mayor, factores que incrementan sustancialmente su complejidad. Además, presentan características (uso múltiple, necesidad de acceso eficiente para análisis, necesidad de indexación, etc.) que hacen esencial el uso de bases de datos y tecnologías específicas para su gestión y administración.

En esta guía el aprendiz desarrollará diferentes actividades tendientes a comprender las diferentes estructuras de almacenamiento de datos existentes y cómo influyen en la disponibilidad, integridad, autenticidad y confidencialidad de los datos.

Palabras claves (key words)

Dato, Estructura, Bit, Sistema Numérico, Almacenamiento de información.

Responde y Dialoga con tu grupo:

- ¿Qué es un dato?
- ¿Qué es información?
- ¿Cómo se puede garantizar la disponibilidad de los datos en la empresa?
- ¿A qué nos referimos con confiabilidad de la información?
- ¿Por qué consideras que la integridad de la información es esencial?
- ¿Qué tanto puede impactar una falla de autenticidad en los datos de una organización?

PLANTEAMIENTO DE LAS ACTIVIDADES

Todos los problemas y situaciones del mundo real son susceptibles de análisis, y a través de dicho análisis pueden ser descompuestos y modelados con el fin de administrarlos de forma más eficiente, automatizarlos y mejorarlos; y todo sistema puede ser descrito en términos de cinco **elementos esenciales**: su contexto, sus procesos, sus entradas, sus salidas y el control que se haga sobre sus resultados.

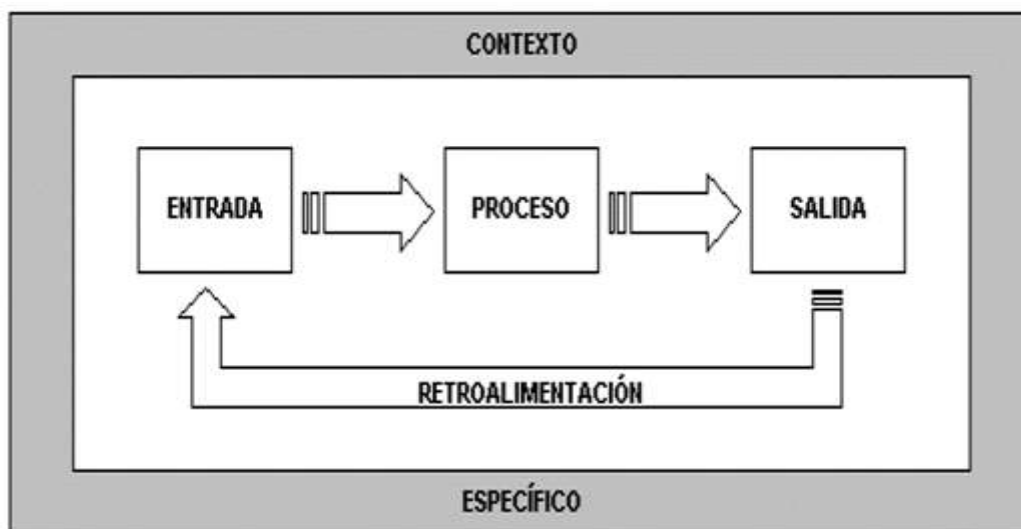


Ilustración 1 Elementos de un Sistema.

Conocer la manera en que las computadoras almacenan los datos es esencial, de otra manera ¿Cómo podríamos gestionarlos y acceder a ellos para analizarlos y tomar decisiones en nuestras organizaciones a partir de la información encontrada?

Las actividades siguientes permiten al aprendiz un acercamiento general a las características de las diferentes estructuras de datos y la forma como impactan el proceso de almacenamiento y recuperación de estos en los sistemas informáticos para sustentar la toma de decisiones en las organizaciones.

Actividad 1: Sistemas Numéricos

Desde las matemáticas se definen los sistemas numéricos como aquellos sistemas de notación que se utilizan para representar cantidades abstraídas del mundo real, las cuales denominamos números. Un sistema numérico está definido por su base. La base es la cantidad de símbolos diferentes, o guarismos¹, necesarios para representar un número cualquiera, de los infinitos posibles, en el sistema.

Un sistema numérico es entonces un conjunto ordenado de símbolos o dígitos, que, mediante reglas definidas, representa cantidades. Una de las características de los sistemas numéricos es que pueden clasificarse en tres grandes grupos: los posicionales, los semi-posicionales y los no posicionales.

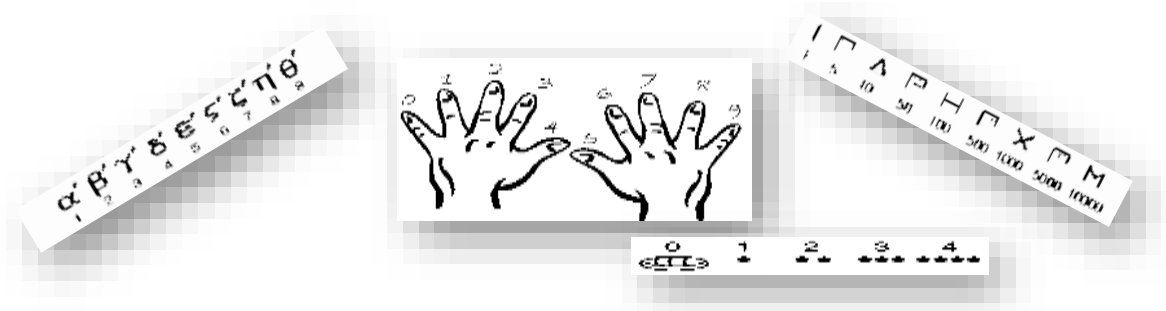


Ilustración 2 - Tipos de Sistemas numéricos.

Para procesar datos se utilizan comúnmente los sistemas numéricos: binario, octal, decimal y hexadecimal.

Base	Sistema	Dígitos
2	Binario	0,1
8	Octal	0,1,2,3,4,5,6,7
10	Decimal	0,1,2,3,4,5,6,7,8,9
16	Hexadecimal	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Ilustración 3 Sistemas numéricos más comunes.

Para poder definir la base en que se encuentra representado un número es necesario expresarlo como un subíndice, a continuación de su magnitud.

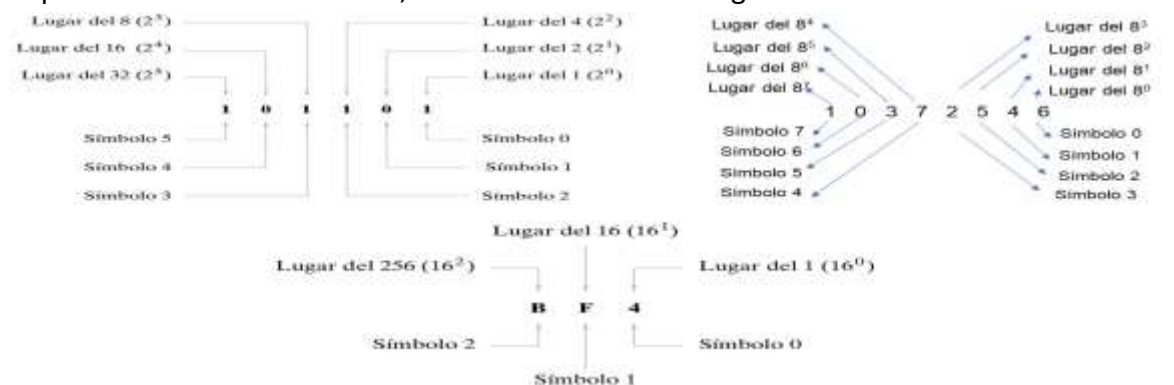


Ilustración 4 Expresión de valor en sistemas numéricos.

Binario

Es el sistema de números que sólo usa como base el 1 y el 0. El 1 en los sistemas de cómputo representa activado y el 0 desactivado, o también alto y bajo respectivamente. Las señales que se crean mediante estas dos condiciones representan dígitos binarios llamados bits. Un

bit es un dígito binario (abreviación del inglés *binary digit*), es decir, un 0 o un 1; la agrupación de 4 bits se conoce como *nibble* y la de 8 bits se conoce como *byte*.

¿Por qué las computadoras utilizan sólo estos dos números?

Porque ellas trabajan internamente con dos estados de voltaje.

En el sistema binario la posición también importa, así, el bit ubicado a la izquierda de un número binario se llama bit de orden superior o bit más significativo (**MSB**, siglas en inglés de *Most Significant Bit*); y el bit a la derecha es el bit de orden inferior o bit menos significativo (**LSB**, siglas en inglés de *Least Significant Bit*).

Octal

El sistema Octal, tiene una base de ocho y utiliza los dígitos para representar las cantidades, estos van del 0 al 7. Al igual que otros, se utiliza a menudo para representaciones breves, relacionadas con números de múltiples bits en el entorno digital. Este sistema numérico, tiene dos ventajas notorias sobre el hexadecimal: no requiere el uso de otros símbolos y la conversión entre binario y octal es muy sencilla.

Decimal

Es el sistema de numeración utilizado naturalmente por las personas en la mayor parte del mundo. Se conoce como Decimal porque está basado en los 10 dedos de la mano, que a su vez son representados con diez símbolos: (0,1,2,3,4,5,6,7,8,9). Dependiendo de la posición que ocupe la cifra, al número se le otorga un valor: unidades, decenas, centenas, millares, entre otros. Y el valor de cada dígito está asociado al de una potencia de base 10, número que coincide con la cantidad de símbolos o dígitos del sistema decimal, y un exponente igual a la posición que ocupa el dígito menos uno, contando desde la derecha.

Hexadecimal

Este sistema se utiliza en las mismas condiciones del octal, es decir para representaciones breves, relacionadas con números de múltiples bits en el entorno digital. Este sistema de base 16, utiliza 16 dígitos: los números del 0 al 9 y las letras (mayúsculas) de la A a la F, todo esto para representar cantidades, el uso que se le da en la actualidad está directamente relacionado con los sistemas de computación. Se denomina hexadecimal, porque Hexa hace alusión a las 6 letras que utiliza y decimal por los caracteres del 0 al 9 (base decimal).

Es importante que no se confunda este sistema con el sexagesimal, el cual emplea como base el 60 y se usa para medir tiempos, coordenadas y ángulos.

Para comprender mejor este tema realice una consulta sobre los sistemas numéricos, sus tipos, características, usos. Identifique la forma en que se aplican dichos sistemas para representar valores o cantidades del mundo real y cómo se hacen conversiones de un sistema numérico a otro.

Elabore una tabla en la cual simbolice las magnitudes siguientes en sistema numérico decimal y su correspondencia en los sistemas numéricos descritos:

- | | | |
|--------|-------|-------|
| • 5050 | • 680 | • 780 |
| • 4096 | • 512 | • 912 |
| • 4567 | • 300 | • 360 |
| • 3800 | • 256 | • 556 |
| • 1098 | • 128 | • 428 |
| • 1024 | • 190 | • 990 |
| • 2048 | • 64 | • 604 |
| • 2512 | • 56 | • 506 |
| • 1970 | • 10 | • 100 |

Consulta:

¿Qué operaciones aritméticas pueden ejecutarse con los sistemas numéricos?

¿Cómo se hacen conversiones de un sistema a otro: binario a octal, octal a decimal, decimal a hexadecimal, etc.?

¿Qué son: bit, nibble, byte?

Actividad 2: Almacenamiento de Datos

¿Cómo utilizar la memoria y el tiempo de ejecución eficientemente con estructuras de datos apropiadas según los requerimientos y recursos disponibles al momento de desarrollar una aplicación?

Sabemos que un **algoritmo** es un conjunto de reglas para resolver un problema. Su ejecución en sistemas de cómputo requiere de recursos, por lo que un algoritmo es mejor cuantos menos recursos consuma. Pero, también dependerá de otros criterios: facilidad de programarlo, corto, fácil de entender, robusto, reutilizable, etc. Esto es lo que entendemos como **Eficiencia**: la relación entre los recursos consumidos y los productos conseguidos.

Eficiencia de las estructuras de datos

Entre los criterios a tener en cuenta al diseñar o elegir un algoritmo están su complejidad, y su tiempo de ejecución. El tiempo de ejecución depende de factores variados y, muy en particular, del tamaño del problema. El tiempo de ejecución no siempre puede ser exacto: es muy difícil de predecir; normalmente sólo se puede saber midiéndolo; la predicción del ritmo de crecimiento del tiempo de ejecución con respecto al tamaño del problema se realiza a través de diferentes métodos matemáticos y estadísticos.

El **manejo de memoria** consiste en la asignación y liberación de bloques de memoria a medida que un programa lo solicita. Particularmente, la liberación de memoria puede realizarse en forma manual o mediante técnicas automatizadas, también conocidas como técnicas de recolección de basura, pues consisten en identificar y liberar los bloques asignados que ya no serán accedidos desde la aplicación (**Simsek, 2005**). Estas técnicas automatizadas pueden clasificarse en dos grupos: basados en rastreo y basados en conteo de referencias. El rastreo de referencias es muy costoso, pero resulta aplicable en cualquier situación, mientras que el conteo de referencias es un esquema mucho más simple pero falla cuando existen referencias circulares, porque ningún contador cae a cero.

Memoria estática

Es el espacio en memoria que se crea al declarar variables de cualquier tipo de dato (primitivas [int, char...] o definidos por el usuario [matrices, struct, enum,...]). La memoria que estas variables ocupan no puede cambiarse durante la ejecución y tampoco puede ser liberada manualmente.

Consideraciones

- ✓ Error en tiempo de ejecución de índice fuera de rango.
- ✓ Se debe conocer con anticipación el tamaño de la estructura.
- ✓ Se guardan en memoria adyacente.
- ✓ Vectores, Matrices, Cubos, Registros, Archivos.

Memoria dinámica

Es memoria que se reserva en tiempo de ejecución. Su principal ventaja frente a la estática es que su tamaño puede variar durante la ejecución del programa. El uso de memoria dinámica es necesario cuando a priori no conocemos el número de datos/elementos a tratar.

Consideraciones

- ✓ Manejo transparente del tamaño de la estructura.
- ✓ Se guarda donde encuentra espacio.
- ✓ No se guardan necesariamente en memoria adyacente.

Puntero

Es una variable que referencia una región de la memoria, tiene un tipo de dato asociado, su valor es una dirección de memoria (formato de localización de bytes de memoria con el cual un programa informático o un dispositivo de hardware accede o almacena datos para su posterior utilización). Para crear cualquier estructura dinámica requiero de apuntadores, los apuntadores permitirán:

- ✓ 'Crear' la ubicación de memoria (*new*).
- ✓ Acceder a los datos en dicha ubicación (*, ^: referenciar, des-referenciar).
- ✓ 'Destruir' los datos en la memoria (*dispose*).
- ✓ Enviar y recibir argumentos y parámetros en funciones.
- ✓ Manejo de cadenas, arreglos.
- ✓ Acceso directo a dispositivos *hardware*.

Una forma común de describir la memoria principal de un ordenador es como una colección de celdas que almacenan datos e instrucciones. Cada celda está identificada unívocamente por un número o dirección de memoria. Para poder acceder a una ubicación específica de la memoria, la CPU genera señales en el bus de dirección, que habitualmente tiene una longitud de 32 bits en la mayoría de máquinas actuales. Un bus de dirección de 32 bits permite especificar hasta $2^{32} = 4.294.967.296$ direcciones de memoria distintas. Las direcciones de memoria se expresan a menudo en código hexadecimal. Por ejemplo, para expresar el valor binario 111111010100000000000010101100 se escribe 0x3F5000AC en hexadecimal.



Ilustración 5 Ejemplo de un apuntador de memoria.

Consulte:

¿Cuál es la estructura de un registro de memoria?

- ¿Cómo se crea un puntero en los lenguajes de programación?
- ¿Qué es un sistema de archivos y cuáles son los más utilizados?
- ¿Cómo se almacenan los datos en un disco duro?

Actividad 3: Estructuras de Datos

En el ámbito de la informática, las estructuras de datos son aquellas que nos permiten, como desarrolladores, organizar la información de manera eficiente, y en definitiva diseñar la solución correcta para un determinado problema.

Ya sean las más utilizadas comúnmente -como las variables, *arrays*, conjuntos o clases- o las diseñadas para un propósito específico -árboles, grafos, tablas, etc.-, una estructura de datos nos permite trabajar en un alto nivel de abstracción almacenando información para luego acceder a ella, modificarla y manipularla.

Estructuras de datos abstractas

Una estructura o tipo de datos abstracto (**ADT**) es una clase o módulo de programa que contiene:

- ✓ Datos privados, con una determinada estructura.
- ✓ Un conjunto de métodos públicos para manejar esos datos.
- ✓ El conjunto de operaciones permite el uso de la estructura de datos sin conocer los detalles de su implementación

Los programas que usan la estructura son independientes de la forma en la que esta se implementa por lo que no es necesario conocer los detalles internos del tipo de datos ni de su implementación. Una Clase es un ente cerrado y autosuficiente que no requiere de un contexto específico para poder ser usada en un programa, garantizando portabilidad y reutilización. Son un modelo de los objetos de datos del mundo real que se necesitan y de las funciones que operan sobre ellos. Los tipos de datos abstractos no están predefinidos, el usuario los define y estructura para poder utilizarlos en el sistema (crear, modificar, analizar, persistir).

¿Cómo se define un objeto(tipo) abstracto?

- Nombre
- Características (atributos y propiedades)
- Restricciones
- Lista de Operaciones (métodos)
- Lista de Servicios (funciones)

Encapsulamiento

Se dice que la clase encapsula el tipo de datos junto a sus operaciones, ocultando los detalles internos.

- ✓ Consiguen la reutilización de código
- ✓ Para muy diversas aplicaciones
- ✓ Aunque necesitamos saber la eficiencia de los métodos.

Estructuras de Datos Estáticas

Las estructuras de datos estáticas son aquellas en las que el tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa. Se trata de una colección de datos que pueden ser caracterizados por su organización y las operaciones que se definen en ella.

Estructuras de Datos Dinámicas

Son estructuras cuya dimensión puede crecer o disminuir durante la ejecución del programa de manera dinámica. Una estructura dinámica de datos es una colección de elementos llamados nodos. Los objetos dinámicos exponen miembros como propiedades y métodos en tiempo de ejecución, en lugar de en tiempo de compilación. Esto permite crear objetos para trabajar con estructuras que no coinciden con un formato o tipo estático.

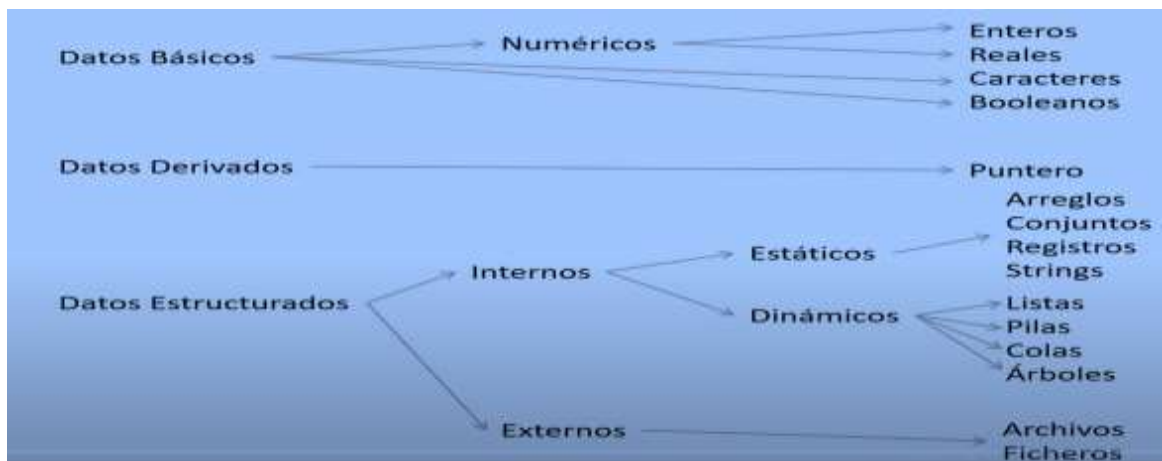


Ilustración 6 Estructuras de datos.

Consulta:

¿Cuáles son los tipos de dato que existen en lenguajes de programación?

¿Cuáles son los tipos de dato que existen en las bases de datos?

¿Qué es un Collation en base de datos?

GLOSARIO

- **Colas:** Es una forma de acceder los datos, contrario a la pila, se elimina (o desencola) por el frente y se ingresan los datos o se encolan por el final, se asemeja a las colas que observamos en el mundo real, en una cola en un banco, se atiende al primero que llega (el que está en el frente) y las personas ingresan al final de la cola.
- **Índice:** Es la posición de un elemento en la matriz, si el elemento está en la fila 2 y columna 1, decimos que la posición es (2,1), si el elemento está en la fila 1 columna 3 decimos que la posición es (1,3)
- **Lista doblemente enlazada:** Es un conjunto de nodos contiguos, que se les va asignando memoria a medida que se van creando, se enlazan entre ellos a través de dos punteros, Anterior: que apunta a la dirección del nodo anterior y Siguiente: que apunta al siguiente nodo.
- **Lista enlazada:** Es un conjunto de nodos, contiguos, a los que se les va asignando memoria a medida que se van creando, se enlazan entre ellos a través de un puntero que contiene la dirección al siguiente nodo. Cuando tiene un solo puntero se dice que es una lista enlazada simple.
- **Matriz:** Es una estructura que permite guardar un grupo de datos del mismo tipo en forma organizada. la matriz más sencilla tiene dos dimensiones conformada por filas y columnas
- **Memoria:** Es un espacio de almacenamiento lógico para almacenar información en el computador durante determinado tiempo. La forma de representar esa información en memoria es a través del sistema binario de 1s y 0s.
- **Memoria dinámica:** Es cuando el tamaño, es decir, la cantidad de bytes que se requieren para almacenar un objeto (por ejemplo las listas enlazadas) es definido en tiempo de ejecución, a medida que el proceso necesite más espacio, va solicitando más memoria al sistema operativo
- **Memoria estática:** Es cuando el tamaño, es decir, la cantidad de bytes que se requieren para almacenar un objeto (por ejemplo un vector o una matriz) es definido en tiempo de compilación, por lo que en la ejecución del programa ya se conoce ese tamaño y no puede modificarse.
- **Nodo:** Es un elemento de una lista enlazada, está conformado por dos partes: Dato que es el espacio donde se guarda la información y Enlace que es el puntero que tiene la dirección al siguiente nodo.
- **Pila:** Es una estructura de datos que está clasificada como una lista lineal especial, por que la inserción (Apilar) y eliminación (Desapilar) de sus datos se realiza solo por la cima o tope.
- **LIFO:** (Último en entrar, primero en salir) se puede comparar con una lista de platos en el que el último que se enjabona, es el primero que se lava.

EVIDENCIAS A PRESENTAR

Evidencias por desempeño

- ✓ *Desarrollo completo de las actividades planteadas en la presente guía.*

Evidencias de conocimiento

- ✓ *Informe en formato PDF con las respuestas a cada una de las preguntas planteadas en la presente guía.*

Evidencias de producto

- ✓ *Informe en formato PDF con las evidencias gráficas de los procesos ejecutados para cumplir con las actividades de la guía y sus correspondientes observaciones, descripciones y/o conclusiones.*

REFERENTES BIBLIOGRÁFICOS

- ✓ Joyanes, L., Fernández, M., Sánchez, L., Zahonero, I., (2005). Estructuras de datos en C. McGraw-Hill. ISBN 8448145127.
- ✓ Godoy, R., María, M., Cardona, A. (2006). Manejo Dinámico De Memoria En Un Programa De Elementos Finitos Orientado A Objetos. Mecánica Computacional Vol XXV, pp. 961-975. Santa Fe, Argentina.

Cibergrafía

- ✓ <https://ootips.org> sitio sobre programación orientada a objetos. Consultado en Octubre 10 de 2022.
- ✓ <https://learn.microsoft.com> librería especializada de herramientas de Microsoft. Consultado en Octubre 10 de 2022.
- ✓ <https://www.mongodb.com/docs> sitio oficial de MongoDB Inc. Consultado en Octubre 10 de 2022.
- ✓ <https://learn.microsoft.com/es-es/sql/sql-server> sitio oficial de la documentación para Microsoft SQL Server. Consultado en Octubre 10 de 2022.
- ✓ <https://docs.microsoft.com/en-us/dotnet/standard/automatic-memory-management> sitio oficial de la documentación para Microsoft SQL Server. Consultado en Octubre 10 de 2022.

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	Luis Fernando Sánchez Pérez.	Instructor	CTMA/TICS- Electrónica.	20/4/2025

CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)	Luis Fernando Sánchez Pérez	Instructor	CTMA/TICS- Electrónica.	9/5/2025	Modificación de Actividades de Apropiación del conocimiento